Assignment 2: Recurrent Models and Transformers

Results draft — empirical runs pending

Naveen Kumar
Student ID: 2026AJ05319

Status. This report is a transparent draft, not a completed experimental submission. The local environment passed the quick PyTorch unit checks, but the full model-training experiments have not been run on the specified T4. No measured accuracy, loss, run time, or generated sample is claimed below. Run both notebooks on the specified T4; then replace pending entries and include the produced figures before submitting.

1 Problem 1: recurrent sequence models

1.1 Data and method

The HAR input has 128 time steps and nine inertial-signal channels. Validation subjects are selected from the official training subjects with `default_rng(seed)`; test subjects remain the official held-out set. Per-channel normalization statistics are computed over the training partition only. The adding data follows the supplied generator: each sequence has a value channel and exactly two markers, one in each half, and the target sums the marked values. Data-generator assertions test all three conditions.

The vanilla RNN, LSTM, and GRU cells use affine transforms and nonlinearities, unrolled over time. Their outputs are checked against the corresponding single-layer PyTorch modules at tolerance 10^{-5}. With input dimension $d = 9$ and hidden dimension $h = 64$, the hand counts with one bias per transform are $hd + h^2 + h = 4{,}736$, $4(hd + h^2 + h) = 18{,}944$, and $3(hd + h^2 + h) = 14{,}208$. PyTorch stores separate input and recurrent biases, giving 4,800, 19,200, and 14,400 parameters, respectively.

1.2 HAR and adding results

Table 1: HAR test results (mean ± standard deviation over three seeds). Populate from `results.csv`.

Model	Accuracy	Macro-F1	Parameters	Seconds/epoch
RNN	pending	pending	pending	pending
LSTM	pending	pending	pending	pending
GRU	pending	pending	pending	pending

The adding sweep uses $T \in \{50, 100, 200, 400\}$, three models, and three seeds. The constant-prediction reference MSE is approximately 0.167; a run is called solved only when test MSE is below 0.05. Mean and standard deviation, solved-run counts, the $T = 200$ learning curves, and MSE-versus-length plot are pending execution. The notebook also creates the class-balance and best-model confusion-matrix figures.

1.3 Gradient and ablation analysis

The required plots are generated by the notebook: hidden-state gradient norm by time step, hidden-size sweep, and a controlled gradient-clipping ablation for the LSTM adding task at $T = 200$. The provided excerpt specified the rubric for this individual ablation but not its exact intervention; gradient clipping is used because it directly tests the gradient-stability intervention. Interpretations

and wearable-device recommendation must be written after seeing the measured accuracy, parameter count, and epoch-time trade-offs. These conclusions are intentionally not fabricated in this draft.

2 Problem 2: Transformer

2.1 Attention and encoder block

Scaled dot-product attention is implemented as $QK^\top/\sqrt{d_k}$, softmax, and the weighted sum of values, with an optional allowed-position mask. Assertions check row sums, agreement with PyTorch scaled dot-product attention, causal future-zero weights, and a three-token hand example. Multi-head attention uses separate learned projections and explicit head reshaping; the encoder is pre-LayerNorm with residual connections, dropout, and a width-$4d$ feed-forward network.

The block's weight-only count is $4d^2 + 2dd_{\text{ff}}$. At $d = 128$, $d_{\text{ff}} = 512$, this is 196,608. The count reconciliation additionally includes projection/feed-forward biases and two LayerNorm affine pairs, and is asserted against `sum(p.numel())`.

2.2 HAR classifier and character model

The HAR encoder classifier compares no positional encoding, sinusoidal encoding, and learned positions using the same subject-wise data pipeline. Results versus the best recurrent model are pending. The character-level model uses Tiny Shakespeare, an ordered 90/10 character split, context 128, $d = 128$, four heads, and four layers. Validation loss in nats, bits per character (nats divided by $\ln 2$), and samples at temperatures 0.2, 0.8, and 1.5 are pending training.

The notebook generates two first-layer attention heatmaps, a 1/4/8-head ablation, a causal-versus-unmasked comparison, and context-length runtime/memory measurements at 64, 128, 256, and 512. The unmasked run sees future characters and is a leakage demonstration, not a valid causal language model. Attention-score storage grows quadratically with context length; measured device behavior should be discussed alongside this complexity.

AI-use statement

AI assistance was used to draft the implementation structure and tests. The student must inspect and run the code, validate the measurements, interpret the plots, and take responsibility for the final submitted work.

References

1. UCI Machine Learning Repository, *Human Activity Recognition Using Smartphones*, `https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones`.

2. Karpathy, A., *tiny Shakespeare*, `https://github.com/karpathy/char-rnn/blob/master/data/tinyshakespeare/input.txt`.

3. Vaswani, A. et al., *Attention Is All You Need*, NeurIPS, 2017.